

Guía de Uso: Evaluador Automático (Grader)

Este evaluador automático (Grader) está diseñado para probar sus soluciones algorítmicas de forma local antes de la entrega final. El programa inyectará automáticamente los casos de prueba ocultos en su código y comparará los resultados para decirles si su lógica es correcta.

Requisitos Previos

El evaluador soporta **C++**, **Python**, **C#** y **Rust**. Dependiendo del lenguaje que elijan para resolver los problemas, asegúrense de tener instalado el compilador o intérprete correspondiente en su sistema:

- **Python:** Python 3 instalado y agregado al PATH (`python` o `python3`).
- **C++:** GCC (`g++`) en Linux/Windows, o Visual Studio (ejecutando el evaluador desde el *Developer Command Prompt*).
- **C#:** .NET SDK instalado (`dotnet`).
- **Rust:** Compilador de Rust instalado (`rustc`).

Estructura de los Problemas

Tienen a su disposición tres problemas para evaluar:

1. `lis` (Longest Increasing Subsequence)
2. `lcs` (Longest Common Subsequence)
3. `party` (Planning a Company Party)

Para cada problema, encontrarán **plantillas (templates)** en los distintos lenguajes.

Instrucciones de Uso

Paso 1: Escribir el código

Abran el archivo de plantilla del lenguaje de su preferencia (ej. `solution.py` o `solution.cpp`). **IMPORTANTE:** Escriban su lógica **únicamente** dentro de la función indicada. No modifiquen la parte del código que lee la entrada de la consola o escribe en el archivo `solution.txt`. El evaluador depende de esa estructura para funcionar correctamente.

Paso 2: Ejecutar el evaluador

Abran su terminal o línea de comandos, naveguen hasta la carpeta donde se encuentra el evaluador y ejecuten el comando con la siguiente estructura:

En Windows:

```
grader.exe <problema> <ruta_al_archivo_de_solucion>
```

En Linux: Primero, denle permisos de ejecución al evaluador (solo se hace una vez):

```
chmod +x grader.out
```

Luego, ejecútenlo:

```
./grader.out <problema> <ruta_al_archivo_de_solucion>
```

(Nota para Windows: Los comandos de ejemplo de abajo usan la sintaxis de Linux. Si usan Windows, simplemente cambien ./grader.out por grader.exe).

Ejemplos de Ejecución por Lenguaje

Supongamos que van a evaluar el problema de la fiesta de la compañía (party).

- **Para evaluar Python:**

```
./grader.out party "ruta/al/archivo/solution.py"
```

- **Para evaluar C++:**

```
./grader.out party "ruta/al/archivo/solution.cpp"
```

(El evaluador compilará automáticamente el código por ustedes). [Unico ejemplo por defecto pensado para windows] * **Para evaluar C#:**

```
grader.exe party "ruta/al/archivo/solution.cs"
```

(No necesitan crear un proyecto en C#, el evaluador creará un entorno temporal, ejecutará el código y limpiará la basura automáticamente). * **Para evaluar Rust:**

```
./grader.out party "ruta/al/archivo/solution.rs"
```

Entendiendo los Resultados

Al ejecutar el evaluador, verán en la consola el progreso de los 10 casos de prueba por cada problema:

- **[PASSED]:** Su programa generó la respuesta correcta para ese caso.
- **[FAILED]:** Su programa generó una respuesta incorrecta. El evaluador les mostrará qué esperaba recibir y qué fue lo que su programa calculó.

Notas Importantes

- **Archivos temporales:** Durante la ejecución, el evaluador creará archivos como `input.txt` y `solution.txt` en su carpeta. No se preocupen por ellos, el evaluador los borrará automáticamente al terminar.
- **Bucles infinitos:** Si su código tiene un bucle infinito (ej. un `while(true)` sin salida), el evaluador se quedará “colgado” esperando. Si esto pasa,

cancelen la ejecución con **Ctrl + C** y revisen su lógica. En caso de tener archivos temporales residuales que no se lograron eliminar por cancelar abruptamente el flujo de ejecución del evaluador borrarlos por favor.

- **C++ en Windows:** Si usan el compilador de Microsoft (**cl.exe**), recuerden abrir el *Developer Command Prompt for VS* para ejecutar el evaluador, de lo contrario no reconocerá los comandos de compilación.

¡Mucho éxito con la implementación de sus algoritmos!